Assignment 1: Assembly

Bar-Ilan CS 89-3312
Parallel Systems Programming

March 25, 2026

Objective

In this assignment, you will implement four tasks that utilize SIMD capabilities, both through assembly and intrinsics. The focus is on performance optimization using SSE / SSE4.2 instructions.

Submission

At the end of the assignment, you will find guidelines on how to submit the code.

Follow the rules and the required file names exactly! If it says to submit `ttttt.c`, then you must upload `ttttt.c` with that exact name.

Task 1: Decode Function Implementation (Warm-up)

Background

For a function,

```
long decode(long x, long y, long z);
```

the following assembly code is generated.

```
{Assembly (x86-64, AT\&T syntax)}
decode:
    subq %rdx, %rsi
    imulq %rsi, %rdi
    movq %rsi, %rax
    salq $63, %rax
    sarq $63, %rax
    xorq %rdi, %rax
    ret
```

Parameters `x`, `y`, and `z` are passed in registers `%rdi`, `%rsi`, and `%rdx`, respectively. The return value is stored in `%rax`.

Requirements

- Implement `decode` in C with the same effect as the given assembly code.
- Save your implementation in a file named `decode.c`.
- Declare the function:

```
long decode_c_version(long x, long y, long z);
```

Task 2: String Length with SSE

Background

The provided `_strlen` function calculates the length of a string in assembly:

```
_strlen:
    pushq %rcx # Save rcx on stack
    xorl %ecx, %ecx # Zero counter (faster than ‘xorq %rcx, %rcx‘)

_strlen_next:
    cmpb $0, (%rdi) # Compare byte at rdi with 0 (null terminator)
    je _strlen_done # If zero, exit loop
    incq %rcx # Increment character count
    incq %rdi # Move to the next character
    jmp _strlen_next # Repeat

_strlen_done:
    movq %rcx, %rax # Move counter to return value (rax)
    popq %rcx # Restore rcx
    ret # Return to caller
```

Requirements

- Implement `strlen_sse42` an optimized version of the `strlen` function using SIMD instructions (specifically SSE) in x86-64 assembly.

Notes

In the lectures, many assembly instructions may have appeared using both Intel and AT&T syntax. As part of this assignment, students are expected to research and understand the correct syntax, operand order, and usage of instructions such as `pcmpistri`, `pcmpistrm`, and other SIMD operations.

Discovering how to use these instructions properly, especially their behavior, operand rules, and control byte configuration (`imm8`)—is a core part of the learning process.

Instructions like `pcmpistri` and `pcmpistrm` require careful setup. Research their operand usage and control byte (`imm8`).

Example AT&T Syntax

```
pcmpistri $imm8, %xmm, %xmm
```

Task 3: Hamming Distance

In this part, you will implement string operations using intrinsics.

Open the `strings` folder using your text editor of choice. The main function runs a function that receives two strings and prints the result of an operation on them.

Your task is to implement the **Hamming Distance** function using intrinsics in C.

Definition

The Hamming Distance of two strings is the number of positions where the corresponding characters are different.

Requirements

- Implement the function using intrinsics.
- If the strings are not the same length, the difference in their lengths should be added to the final Hamming distance.

Implementation Details

- Use intrinsics to compare multiple characters at a time, instead of checking one character at a time in a loop.
- Store the comparison results in a vector register and count the differences efficiently.

Input Example

```
str1 = "hello world"
str2 = "hxllo worl!"
```

Output

```
Hamming Distance = 2
```

Function Signature

```
int hamming_dist(char str1[MAX_STR], char str2[MAX_STR]);
```

File Name

`hamming_intrinsics.c`

Task 4: The Formula

For this part, open the `formulas` folder.

The `main` function runs the functions defined in `formulas.h` on test values and compares their results to the correct results.

Since floating point arithmetic is involved, rounding errors may occur. To compare results, the helper function `is_close` (defined in `main.c`) is used.

Problem Description

Given an array of floating-point numbers x and its length n, compute:

$$\sqrt{1 + \sqrt{\sum_{k=1}^{n} \sqrt{x_k}} \cdot \prod_{k=1}^{n} (x_k^2 + 1)}$$

Requirements

- Implement the function using intrinsics.
- Write the implementation in a file named `formula1.c`.

Function Signature

```
float formula1(float *x, unsigned int length);
```

Submission Guidelines

1. Work can be done individually or in pairs.
2. Submission is through **submit**.
3. Ensure your code compiles and runs without errors or warnings on **BIU servers**.
4. **Deadline: 7.5.2026**
5. In the first line of every submitted file, include a comment with your ID and full name. Example:

   ```
   /* 123456789 Israela Israeli */
   ```

6. Failure to use SSE/AVX (intrinsics or pure assembly) where required will result in an automatic score of 0.
7. Submit a zip file named `ex1.zip` containing:

```
ex1.zip
 decode.c
 strlen_sse42.s
 hamming_intrinsics.c
 formula1.c
```